# NEXUS Manual técnico y preparación para el examen H3

## Para qué sirve esta guía

Esta guía explica cómo está construido NEXUS y cómo razonar sobre sus objetos antes de ejecutar o modificar el programa. Usa como fuente el archivo `Program.cs` incluido en el proyecto. NEXUS es un juego de consola en C# cuyo estado vive en memoria: el cadete explora realidades, usa objetos, encuentra eventos y debe completar una misión.

La lectura del código muestra ejemplos reales de clases, encapsulamiento, herencia, abstracción, polimorfismo y `List<T>`. No hay evidencia de interfaces, LINQ, almacenamiento persistente ni autenticación de usuarios. Cuando esta guía cubre conceptos ausentes, los señala como ejemplos complementarios para estudiar H3.

## 1. Arquitectura y programación orientada a objetos

### Propósito y ejecución general

`Program.Main()` configura la realidad inicial, muestra una introducción, entrega tres objetos iniciales y entra en el ciclo principal. En cada vuelta verifica si el cadete sigue vivo y si la misión continúa activa, presenta estado y menú, y procesa la opción escrita en la terminal (`Program.cs`, líneas 539–740). La opción Explorar ejecuta `Explorar()`, que delega la exploración a la realidad actual y después genera un evento (`Program.cs`, líneas 852–923).

La misión modela sus objetivos mediante propiedades booleanas y solo queda completada cuando se han registrado anomalía, núcleo, enemigo superado y extracción (`Mision`, `Program.cs`, alrededor de líneas 450–535). Las reglas de energía y estabilidad están en `Realidad.Explorar`; el inventario está encapsulado en `Inventario`.

### Responsabilidades y relaciones

| Tipo | Responsabilidad | Evidencia en el proyecto |
|---|---|---|
| `Program` | Coordinar inicio, menú, eventos y misión | `Main`, `Explorar`, `GenerarEvento` |
| `Objeto` | Definir datos comunes y contrato para usar un objeto | Base abstracta; `Usar(Cadete)` |
| `CelulaEnergia`, `Escudo`, `ModuloAtaque`, `Analizador`, `NucleoEnergetico` | Especializar el comportamiento de cada objeto | Sobrescriben `Usar` |
| `Inventario` | Guardar y administrar objetos | Campo privado `List<Objeto> objetos` |
| `Personaje` | Compartir nombre, salud y comportamiento de personaje | Base abstracta |
| `Cadete`, `Robot`, `Enemigo` | Modelar personajes concretos | Derivan de `Personaje` |
| `Realidad` | Mantener riesgo, estabilidad y reglas de exploración | `Explorar(Cadete)` |
| `Mision` | Mantener condiciones de avance y finalización | `Completar...`, `Verificar()` |

El diagrama de clases y el flujograma completos están en `docs/diagramas.md`.

## POO frente a programación estructurada

En un enfoque estructurado, se podría guardar salud, energía e inventario en variables separadas y escribir procedimientos que recibieran todas esas variables. En NEXUS, el estado y las operaciones relacionadas se agrupan: `Cadete` tiene salud y energía y puede recibir daño; `Inventario` mantiene su lista privada y expone operaciones para consultarla o modificarla. Esto facilita razonar sobre qué objeto es responsable de cada regla.

La POO no elimina las funciones ni las secuencias de control. `Program` usa bucles y `switch` para organizar el recorrido del juego, mientras delega las reglas a objetos de dominio. En este código conviven ambos estilos.

# Diagramas de arquitectura

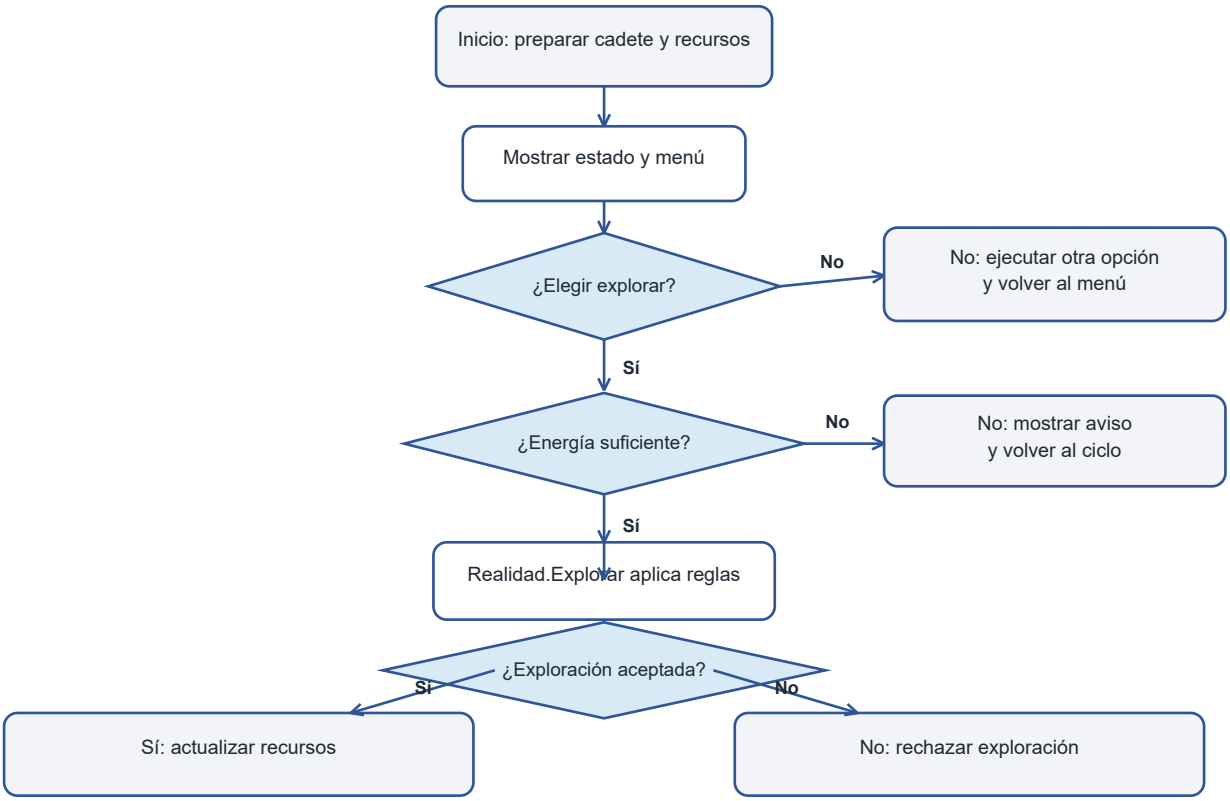
Relaciones principales de las clases. La flecha de herencia apunta a la clase base.

Personaje (abstracto)
Personaje ← Cadete, Robot, Enemigo

Objeto (abstracto)
Objeto ← CelulaEnergia, Escudo, ModuloAtaque, Analizador, NucleoEnergetico

Cadete contiene Inventario; Inventario administra una List<Objeto>.
Program coordina Cadete, Realidad y Mision.

## Flujo de explorar




Después, el código actual genera un evento y vuelve al menú.

# 2. Clases, objetos y encapsulamiento

## Clases e instanciación

Una clase describe los datos y operaciones posibles; un objeto es una instancia concreta creada en tiempo de ejecución. Por ejemplo, `new Cadete("NEXUS-CADETE-01")` crea el objeto cadete y ejecuta su constructor. `new Inventario()` crea el inventario que queda asociado con ese cadete. En `Main` también se crean objetos iniciales con `new CelulaEnergia()`, `new Escudo()` y `new Analizador()`.

## Campos, propiedades y métodos

- Un campo guarda estado directamente. `Inventario` tiene el campo privado `objetos` (`Program.cs`, línea 140); `Program` mantiene campos estáticos como `sesionActiva`.
- Una propiedad ofrece acceso controlado a un valor. `Cadete.XP` usa `private set`: se puede leer públicamente, pero solo `Cadete` puede asignarlo. `Inventario` usa `private set` para impedir que código externo reemplace la referencia al inventario.
- Un método define una operación. `Agregar`, `Mostrar`, `Obtener` y `Eliminar` son métodos de `Inventario`; `Explorar` es un método de `Realidad`.

`public` permite acceso desde cualquier código que pueda ver el tipo; `private` lo limita a la clase; `protected` permite acceso a la clase y a sus derivadas. Por ejemplo, `Objeto.Nombre` tiene `protected set`, así las subclases pueden inicializarlo a través del constructor base, pero el resto del programa no puede cambiarlo directamente. `Inventario.objetos` es privado.

Encapsular no significa esconder todo: significa proteger invariantes y ofrecer operaciones claras. La lista no se expone para que cualquier método externo pueda insertar valores sin pasar por `Agregar`.

## Constructores, sobrecarga, `this` y `base`

El constructor prepara el objeto en un estado válido. `Cadete(string nombre)` inicializa la salud mediante `base(nombre, 100)`, establece energía, estabilidad, escudo y XP, y crea el inventario (`Program.cs`, líneas 244–254). `CelulaEnergia()` llama al constructor protegido de `Objeto` con nombre, tipo y descripción.

La sobrecarga ocurre cuando una misma clase declara varios métodos o constructores con el mismo nombre y diferentes listas de parámetros. En `Enemigo`, por ejemplo, solo hay un constructor con tres parámetros; que otras clases tengan constructores distintos no significa que haya sobrecarga dentro de `Enemigo`. Se puede estudiar una sobrecarga complementaria así:

```
class Enemigo
{
    public string Nombre { get; }
    public int Salud { get; }
    public int Dano { get; }

    public Enemigo(string nombre) : this(nombre, 80, 12) { }
    public Enemigo(string nombre, int salud, int dano)
    {
        Nombre = nombre;
        Salud = salud;
        Dano = dano;
    }
}
```

`this(...)` delega a otro constructor de la misma clase; `this.Propiedad` identifica un miembro del objeto actual. `base(...)` llama al constructor de la clase base. En el proyecto sí se usa `base(...)` en `Cadete`, `Robot`, `Enemigo` y las clases concretas de `Objeto`; no se usa explícitamente `this`.

# 3. Herencia, abstracción y polimorfismo

## Clases base y derivadas

`Personaje` concentra nombre, salud, `EstaVivo()` y la operación de recibir daño. `Cadete`, `Robot` y `Enemigo` heredan ese estado común. Cada derivada aporta reglas propias: el cadete tiene energía e inventario; el robot tiene defensa; el enemigo tiene daño de ataque.

`Objeto` concentra nombre, tipo y descripción, y obliga a cada subtipo a definir `Usar(Cadete)`. Una célula de energía suma 25 de energía con tope 100; un escudo añade 30 de protección con tope 100; el módulo aumenta el daño próximo (`Program.cs`, líneas 18–136).

### `abstract`, `virtual` y `override`

- Una clase `abstract` sirve de base y no se instancia directamente. `Personaje` y `Objeto` son abstractas.
- Un método `abstract` declara una operación que las derivadas concretas deben implementar. `Objeto.Usar` y `Personaje.MostrarInformacion` son abstractos.
- Un método `virtual` proporciona una implementación que una derivada puede redefinir. `Personaje.RecibirDanio` descuenta salud y limita el resultado a cero.
- `override` implementa o reemplaza una operación heredada. `Cadete`, `Robot` y `Enemigo` sobrescriben `MostrarInformacion`; los objetos concretos sobrescriben `Usar`.

## Polimorfismo

Una variable de tipo base puede referirse a objetos de varias subclases. El inventario guarda todos los objetos en `List<Objeto>`, aunque en tiempo de ejecución haya una `CelulaEnergia`, un `Escudo` u otro tipo. Al ejecutar `objeto.Usar(cadete)`, C# selecciona la implementación sobrescrita correspondiente al tipo real del objeto. El llamador puede tratar los elementos de manera uniforme sin escribir un `switch` para cada operación.

En cambio, el método de recibir daño del enemigo no tiene override en `Enemigo`; la llamada usa la implementación heredada de `Personaje.RecibirDanio`. Distinguir qué método se sobrescribe y cuál se hereda es parte del seguimiento de una llamada polimórfica.

## Interfaces, herencia y composición

Una interfaz define un contrato que tipos no relacionados pueden implementar. No aparece una declaración `interface` en el código entregado. Ejemplo complementario:

```
interface IAnalizable
{
    string ObtenerAnalisis();
}

class EnemigoAnalizable : Enemigo, IAnalizable
{
    public EnemigoAnalizable(string nombre, int salud, int dano)
        : base(nombre, salud, dano) { }

    public string ObtenerAnalisis() => "Debilidad: ataque directo";
}
```

La herencia expresa una relación “es un”: `Cadete` es un `Personaje`. La composición expresa “tiene un”: `Cadete` tiene un `Inventario`. Una interfaz expresa “puede hacer”: cualquier clase que implemente `IAnalizable` promete ofrecer ese comportamiento. Favorece capacidades compartidas sin obligar a una jerarquía común.

# 4. Colecciones y LINQ

## `List<T>` y recorrido tradicional

`Inventario` declara `private List<Objeto> objetos` e inicializa una lista vacía. `Agregar` usa `Add`; `Mostrar` recorre por índice con `for`; `TieneTipo` usa `foreach` y devuelve `true` cuando encuentra el tipo pedido. `Obtener` comprueba los límites antes de devolver el elemento, y `Eliminar` delega en `Remove` (`Program.cs`, líneas 138–204).

`List<T>` es una colección genérica: `T` indica qué tipo de elemento se permite. En este caso, `List<Objeto>` acepta instancias de `Objeto` o sus clases derivadas. El tipo genérico da comprobación de tipos en compilación y permite usar miembros del tipo declarado.

## LINQ: ejemplo complementario

No hay llamadas LINQ en `Program.cs`. Si se dispone de una secuencia `IEnumerable<Objeto> objetos`, podrían escribirse:

```
var analizables = objetos
    .Where(o => o.Tipo == TipoObjeto.Analizador);

var nombres = objetos.Select(o => o.Nombre);

var ordenados = objetos.OrderBy(o => o.Nombre);
```

`Where` filtra elementos, `Select` transforma cada elemento y `OrderBy` ordena la secuencia por una clave. La expresión `o => o.Nombre` es una lambda: recibe `o` y produce el valor que se usa para proyectar u ordenar. Para saber si existe alguno que cumpla una condición se puede usar `Any`.

La alternativa tradicional es crear una lista de resultados y llenarla con `foreach`. El bucle deja cada paso explícito; LINQ suele expresar directamente la intención. Ambos enfoques recorren los datos y el resultado debe verificarse. Algunas operaciones LINQ son diferidas: el filtrado puede ejecutarse cuando se enumera la consulta, no al declarar la variable. `ToList()` fuerza una materialización si se necesita guardar una copia.

# 5. Análisis del funcionamiento

## Flujograma de exploración

Ver `docs/diagramas.md`, apartado “Flujo de exploración”. El proceso empieza al seleccionar explorar, comprueba energía, delega al objeto `Realidad`, actualiza recursos según las condiciones y genera un evento aleatorio. El diagrama señala también un detalle importante: `Program.Explorar()` llama a `GenerarEvento()` después de la llamada a `Realidad.Explorar()`, aunque no sabe si esa exploración fue aceptada.

## Seguimiento del estado de un objeto

Caso verificable: cadete con energía 82 y una célula de energía en el inventario. Al usarla, `CelulaEnergia.Usar` guarda `anterior = 82`, suma 25 y aplica el límite 100; por tanto, el valor final es 100, no 107. `UsarObjeto()` consume el objeto si su tipo es célula, escudo o módulo de ataque. El inventario pierde esa célula, y el cadete queda con energía 100. Si la energía ya era 100, el método imprime que está al máximo y retorna sin alterar la energía; `UsarObjeto` igualmente elimina la célula tras el retorno. Ese comportamiento puede ser discutible como regla de juego.

## Predicción de código

```
var c = new Cadete("A");
c.Energia = 80;
new CelulaEnergia().Usar(c);
Console.WriteLine(c.Energia);
```

Resultado esperado: `100`. El constructor empieza con energía 100, luego se cambia a 80. `Usar` suma 25 (105) y el condicional lo limita a 100.

### ¿Qué pasa si se modifica una condición?

En `Realidad.Explorar`, la exploración segura requiere energía y estabilidad de al menos 70; la inestable requiere ambas de al menos 40. Si se cambia el umbral seguro de 70 a 60, cadetes con valores entre 60 y 69 entrarán en el caso seguro y pagarán el costo bajo (energía `10 + NivelRiesgo`, y 3 o 5 puntos de estabilidad), en vez del caso inestable. El cambio altera balance y resultados, no solo el texto mostrado.

Otra mejora sería cambiar `Realidad.Explorar` para que devuelva `bool` o un resultado de exploración. Entonces `Program.Explorar()` generaría evento solo si el resultado fue aceptado. Esto hace explícita la dependencia entre ambos métodos y evita que aparezca un evento tras una exploración rechazada.

# 6. Preguntas de práctica con respuestas razonadas

## 1. Comprensión de POO

Pregunta: ¿Por qué `Inventario` mantiene `objetos` privado en vez de exponer directamente la lista pública?

Respuesta: Para conservar control sobre cómo se modifica el inventario. Los llamadores usan `Agregar` y `Eliminar`, y las consultas pasan por `Mostrar`, `Obtener`, `Cantidad` o `TieneTipo`. Si la lista fuera pública, cualquier código podría borrar elementos o insertar valores en momentos que rompan las reglas del juego.

## 2. Clases, objetos y constructores

Pregunta: ¿Qué estado inicial recibe `new Cadete("Luz")` y qué trabajo realiza `base(nombre, 100)`?

Respuesta: El constructor de `Personaje` asigna nombre Luz y salud 100. Después el constructor de `Cadete` establece energía y estabilidad en 100, escudo en 50, XP y daño extra en 0, y crea un inventario vacío. `base(...)` inicializa la parte heredada del objeto antes de ejecutar el cuerpo del constructor de `Cadete`.

## 3. Herencia y polimorfismo

Pregunta: Una variable `Objeto x = new Escudo();`. ¿Qué implementación corre al invocar `x.Usar(cadete)`? ¿Por qué?

Respuesta: Corre `Escudo.Usar`, porque es el tipo real del objeto y el método está sobrescrito con `override`. El tipo de la variable permite invocar el contrato declarado por la clase base; la resolución dinámica selecciona la implementación derivada.

## 4. Interfaces y diseño

Pregunta: Si tanto `Robot` como `Enemigo` pudieran analizarse, pero no comparten una misma clase padre adecuada para esa capacidad, ¿qué diseño complementario sirve y por qué?

Respuesta: Una interfaz como `IAnalizable` con un método `ObtenerAnalisis()`. Cada clase implementaría el contrato según su propio comportamiento. Así el consumidor depende de la capacidad y no necesita una nueva jerarquía artificial. En el NEXUS entregado no está implementada esta interfaz.

## 5. Listas y LINQ

Pregunta: Dada una secuencia `objetos` de `Objeto`, ¿qué devuelve `objetos.Where(o => o.Tipo == TipoObjeto.Escudo).Select(o => o.Nombre)`?

Respuesta: Una secuencia de nombres de los objetos cuyo tipo sea escudo. `Where` conserva coincidencias; `Select` proyecta cada coincidencia a su nombre. No cambia la lista original. Es una consulta complementaria: el código real busca tipos con `foreach` en `TieneTipo`.

## 6. Predicción y modificación

Pregunta: Si un cadete tiene salud 100, escudo 8 y recibe 13 de daño mediante `RecibirDanioCadete(13)`, ¿cómo queda?

Respuesta: El escudo absorbe `min(8,13) = 8`, queda en 0 y resta 8 del daño pendiente. Los 5 restantes reducen la salud a 95. Si se modifica la línea que descuenta el daño pendiente, hay que revisar que no se aplique daño completo además del absorbido.

### 7. Flujo y condición

Pregunta: ¿Puede ocurrir un evento aunque `Realidad.Explorar` rechace la exploración? Explica la ruta de llamadas y propone una corrección.

Respuesta: Sí. `Program.Explorar()` invoca el método y luego llama a `GenerarEvento()` sin comprobar un valor de retorno. `Realidad.Explorar()` retorna anticipadamente si los recursos no permiten la exploración, pero al ser `void` no comunica el rechazo. Puede devolver `bool` y condicionar la generación del evento al resultado verdadero.

### 8. Abstracción y override

Pregunta: ¿Qué ocurriría si `Enemigo` no implementara `MostrarInformacion()`?

Respuesta: Como `Personaje.MostrarInformacion()` es abstracto, una clase derivada concreta debe implementarlo. Si se elimina el override sin declarar `Enemigo` abstracta, el proyecto no compila. Si `Enemigo` se declara abstracta, no podría instanciarse directamente.

## 7. Plan de repaso para el examen

- Dibuja la relación entre `Personaje`, `Cadete`, `Robot` y `Enemigo`; explica qué miembros se heredan y cuáles se agregan.
- Sigue `UsarObjeto()` para una célula con energía 82. Anota el estado antes y después de `Usar`, y después de la eliminación de inventario.
- Explica qué cambia al invocar un método virtual mediante una referencia base y cómo `override` influye en la llamada.
- Escribe una interfaz de capacidad (`IAnalizable`) y justifica qué clases deberían implementarla.
- Convierte el `foreach` de `TieneTipo` a LINQ usando `Any`; explica qué valor devuelve con y sin coincidencias.
- Predice las consecuencias de cambiar los umbrales de energía/estabilidad y diseña una señal de retorno para la exploración aceptada o rechazada.

Antes de memorizar, intenta contestar cada pregunta sin mirar la respuesta. Luego verifica la explicación directamente en `Program.cs`.

## 8. Diagnóstico del proceso de aprendizaje

El siguiente diagnóstico se limita a la evidencia visible en esta conversación. Las explicaciones técnicas del asistente no cuentan como demostración del aprendizaje de la estudiante.

### 1. Fortalezas demostradas

- Identificó el repositorio correcto al proporcionar su enlace de GitHub después de que la búsqueda inicial no permitió distinguirlo. Esto muestra una acción concreta para resolver una referencia ambigua.
- Compartió el enunciado completo de la actividad, lo que permite delimitar el producto, sus temas y los criterios de entrega.

No hay respuestas propias a preguntas de programación ni explicaciones de conceptos en esta conversación, así que no se puede afirmar dominio técnico demostrado.

### 2. Falencias observadas

- No hay errores conceptuales observables para corregir porque la estudiante no explicó todavía los conceptos del Hito 3.
- La evidencia es insuficiente para saber qué parte del código resulta difícil, cómo razona sobre la ejecución o qué conceptos confunde. Esto es falta de evidencia, no prueba de una falencia.

## 3. Diagnóstico del Hito 3

| Concepto | Clasificación | Evidencia |
|---|---|---|
| POO | Evidencia insuficiente | No hubo explicación propia de diseño orientado a objetos. |
| Clases y objetos | Evidencia insuficiente | No hubo respuesta propia que distinga clase de instancia. |
| Constructores | Evidencia insuficiente | No hubo análisis propio de una inicialización. |
| Encapsulamiento | Evidencia insuficiente | No hubo explicación propia de campos, propiedades o accesos. |
| Herencia | Evidencia insuficiente | No hubo interpretación propia de una clase base y derivada. |
| Polimorfismo | Evidencia insuficiente | No hubo predicción propia de una llamada sobrescrita. |
| Interfaces | Evidencia insuficiente | No hubo ejemplo o razonamiento propio sobre contratos. |
| Listas genéricas | Evidencia insuficiente | No hubo análisis propio de `List<T>`. |
| LINQ | Evidencia insuficiente | No hubo uso o explicación propia de consultas LINQ. |

## 4. Forma de aprender

En la conversación se observa que aportó información adicional cuando se le pidió precisar el repositorio. No hay suficiente material para valorar cómo contrasta una explicación con el código, cómo resuelve un error ni cómo corrige su razonamiento.

## 5. Plan de estudio

Los cinco temas prioritarios sugeridos, por ser los centrales del Hito 3 y no contar aún con evidencia individual, son:

- Clases, objetos y constructores: crea tres cadetes con estados iniciales distintos y registra qué asigna cada constructor.
- Encapsulamiento: intenta modificar `XP` e `Inventario` desde `Main`; predice qué compila y explica el papel de `private set`.
- Herencia y polimorfismo: almacena distintos `Objeto` en una lista base y predice cuál `Usar` se invoca para cada elemento.
- Interfaces: implementa `IAnalizable` en dos clases distintas y llama a ambas a través de una colección de esa interfaz.
- Listas y LINQ: filtra objetos por tipo con `Where`, proyecta nombres con `Select`, ordena con `OrderBy` y compara con un `foreach`.

## 6. Conclusión

La evidencia demuestra que pudo identificar el proyecto y aportar el enunciado necesario para comenzar. No permite concluir qué conceptos de H3 domina todavía. Para demostrar preparación, debería poder explicar con sus palabras la inicialización del cadete, seguir una llamada polimórfica, predecir cómo cambia el inventario y escribir o corregir una consulta simple de lista/LINQ sin apoyo.